EXPERIMENT - 6

Automating Business Processes using Deluge Scripting and Workflows, with Application Testing and Debugging

Insurance Claim Request & Processing Management

Application Name	Insurance Claim Request & Processing Management
Form Name	Claim_Request_Form
Experiment Focus	Four workflows, validation, references and processing
Student Name	HARIHARAN N
Student Register No.	241401024
Experiment	6
Platform	Zoho Creator
Course	Low-Code Application Development using Zoho Creator

Lab Record
Deluge and native field behaviour automate validation, claim references, initial team routing and notifications. Actual positive and negative tests demonstrate the resulting behaviour.

Department of Computer Science and Business Systems

Rajalakshmi Engineering College

1. Objective

To automate insurance-claim submission and processing with Deluge, validate business rules, and test/debug the workflows using synthetic records.

2. Business Process

A claimant submits validated information. Creator assigns an Auto Number reference. The submission workflow sets Submitted status and routes the request to a type-based claims team. Staff then review the claim, assign an officer and update its processing status. A status change triggers a notification.

3. Workflow Inventory

Workflow	Trigger	Action
WF_New_Claim_Form	Created - On Load	Default Submitted; hide staff fields on new submission.
WF_Validate_Claim	Created or Edited - On Validate	Check amount, date, declaration, names, mobile and approval constraints.
WF_New_Claim_Submitted	Created - On Success	Initial status/team assignment; call mail API; log response.
WF_Claim_Status_Notification	Edited - Claim_Status update	Send reference/current-status email and log response.

The status workflow link name is WF_Claim_Status_Notificat. The reusable function is Send_Claim_Mail_API(map request_details). Its connection and API configuration are documented in Experiment 7.

4. State and Assignment Design

Supported statuses: Submitted, Under Review, Assigned, Investigation, Approved and Rejected. Initial assignment is Claim_Type + " Claims Team". Staff can replace the team text with an officer name. Claim_ID is generated by an Auto Number field, not by a Deluge counter.

5. Validation Script

Saved Deluge source from WF_Validate_Claim. Long source lines wrap for print.

```
if(input.Claim_Amount_Requested == null || input.Claim_Amount_Requested <= 0)
{
 alert "Claim amount must be greater than zero.";
 cancel submit;
}
if(input.Date_Of_Incident != null && input.Date_Of_Incident > zoho.currentdate)
{
 alert "Date of incident cannot be in the future.";
 cancel submit;
}
if(input.Declaration_Of_Authenticity == null || input.Declaration_Of_Authenticity.isEmpty())
{
 alert "Please agree to the declaration of authenticity.";
 cancel submit;
}
if(ifnull(input.Claimant_Name.first_name,"").trim() == "" ||
    ifnull(input.Claimant_Name.last_name,"").trim() == "")
{
 alert "Please enter both First Name and Last Name.";
 cancel submit;
}
if(input.Approval_Amount != null)
{
 if(input.Approval_Amount < 0 || input.Approval_Amount > input.Claim_Amount_Requested)
 {
  alert "Approval amount must be between zero and the requested claim amount.";
  cancel submit;
 }
}
if(input.Claim_Status == "Approved")
{
 if(ifnull(input.Approval_Amount,0) <= 0)
 {
  alert "Enter a positive approval amount before marking a claim Approved.";
  cancel submit;
 }
}
if(!ifnull(input.Contact_Number,"").replaceAll("
    ","").replaceAll("-","").matches("[+]91[6-9][0-9]{9}"))
{
 alert "Enter a valid Indian mobile number: +91 followed by 10 digits starting with 6, 7, 8 or
    9.";
 cancel submit;
}
```

6. Submission and Status Scripts

WF_New_Claim_Submitted

```
input.Claim_Status = "Submitted";
input.Assigned_Officer = input.Claim_Type + " Claims Team";
mail_request = Map();
mail_request.put("account_id","7228111000000002002");
mail_request.put("from_address","hariharan1807@zohomail.in");
mail_request.put("to_address",ifnull(input.Email_Address,""));
mail_request.put("subject","Claim Received - Ref " + input.Claim_ID);
mail_request.put("content","Your claim request has been received. Reference: " + input.Claim_ID
    + ". Status: Submitted. Please sign in to the Insurance Claim application to view your
    claim. Quote your reference when contacting the Claims Team.");
mail_result = thisapp.Send_Claim_Mail_API(mail_request);
info "Acknowledgement API result for claim " + input.Claim_ID + ": " + mail_result;
```

WF_Claim_Status_Notification

```
mail_request = Map();
mail_request.put("account_id","7228111000000002002");
mail_request.put("from_address","hariharan1807@zohomail.in");
mail_request.put("to_address",ifnull(input.Email_Address,""));
mail_request.put("subject","Claim Status Update - Ref " + input.Claim_ID);
mail_request.put("content","Your claim reference " + input.Claim_ID + " has been updated.
    Current status: " + input.Claim_Status + ". Please sign in to the Insurance Claim
    application for details or contact the Claims Team with your reference.");
mail_result = thisapp.Send_Claim_Mail_API(mail_request);
info "Status notification API result for claim " + input.Claim_ID + ": " + mail_result;
```

7. Implementation and Debugging Procedure

1. Configure the Created/On Load workflow to hide Claim Processing fields from new submissions.

2. Add server-side validation with alert and cancel submit for rejected conditions.

3. Use native Claim_ID generation; initialize status and type-based routing after successful creation.

4. Call the shared mail function from submission and status-change workflows. Inspect info logs for success/code values.

5. Create synthetic Motor, Health and Property records, then exercise amount, date, contact and approval rejection cases.

6. Process demo records through Under Review and Approved; verify reports, approval totals and notification contents.

8. Negative Test Results

Test input	Observed result
Missing required fields / declaration	Submission blocked; required-field errors displayed.
Malformed email	Email format error displayed.
Phone 123	Indian mobile validation alert; no record created.
Requested amount 0	Claim amount must be greater than zero.
Future incident date 10-Oct-2026 during 9-Oct test	Date of incident cannot be in the future.
Approved with blank approval amount	Positive approval amount required.
Approval 26,000 for request 25,000	Approval amount limit alert.
Approval 20,000 for request 25,000	Accepted; claim #9 saved as Approved.

9. Positive Tests and Debugging Observations

Scenario	Observed result
New demo claim #9	Reference generated; Submitted status and Travel Insurance Claims Team assigned. Acknowledgement API code 200.
Approve demo claim #9	Approved with INR 20,000. Correct status email received; excluded from pending report.
Officer test: claim #10	Caleb changed status to Under Review and assigned Caleb (Demo Claims Officer). Status email delivered.
Policyholder test: claim #11	Authenticated submission saved; only own claim visible. Acknowledgement marked Delivered.
Mail failure / guard handling	Blank or example.com recipients are skipped. Missing configuration returns a message. Exceptions are caught; saved claim data remains available.

Debugging included correcting field links and constraints, removing unintended unique-email/phone restrictions, and persisting the officer permission settings before repeating the affected checks. Workflows were verified through actual records, validation alerts, API logs and mail delivery evidence.

All approval/status tests used explicitly labelled demonstration records. Existing unrelated records and applications were left unchanged.

This is a working lab implementation: statuses are selected by authorized staff; no separate multi-level approval engine or enforced transition graph is claimed.

10. Implementation Evidence - Workflow List

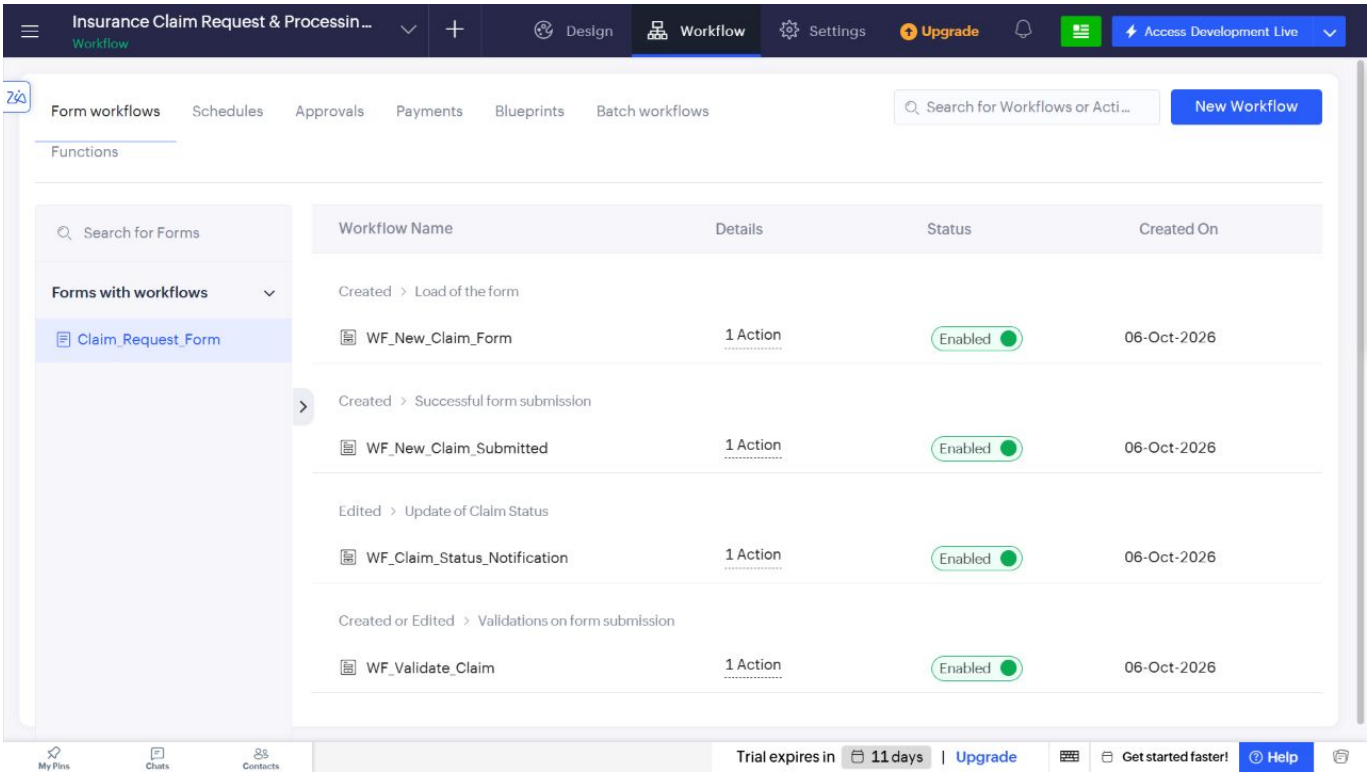


Figure 1. Actual configured workflow list with the four enabled form workflows.

10. Implementation Evidence - Deluge Editor

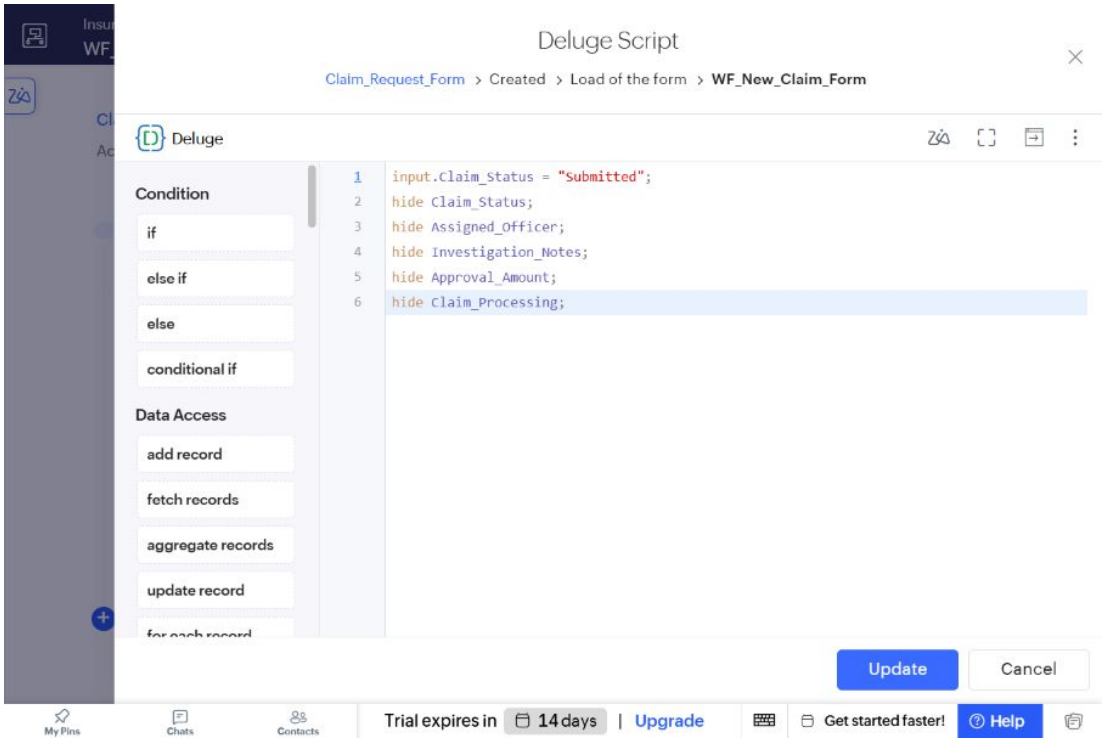


Figure 2. On-load Deluge hiding staff-processing fields.

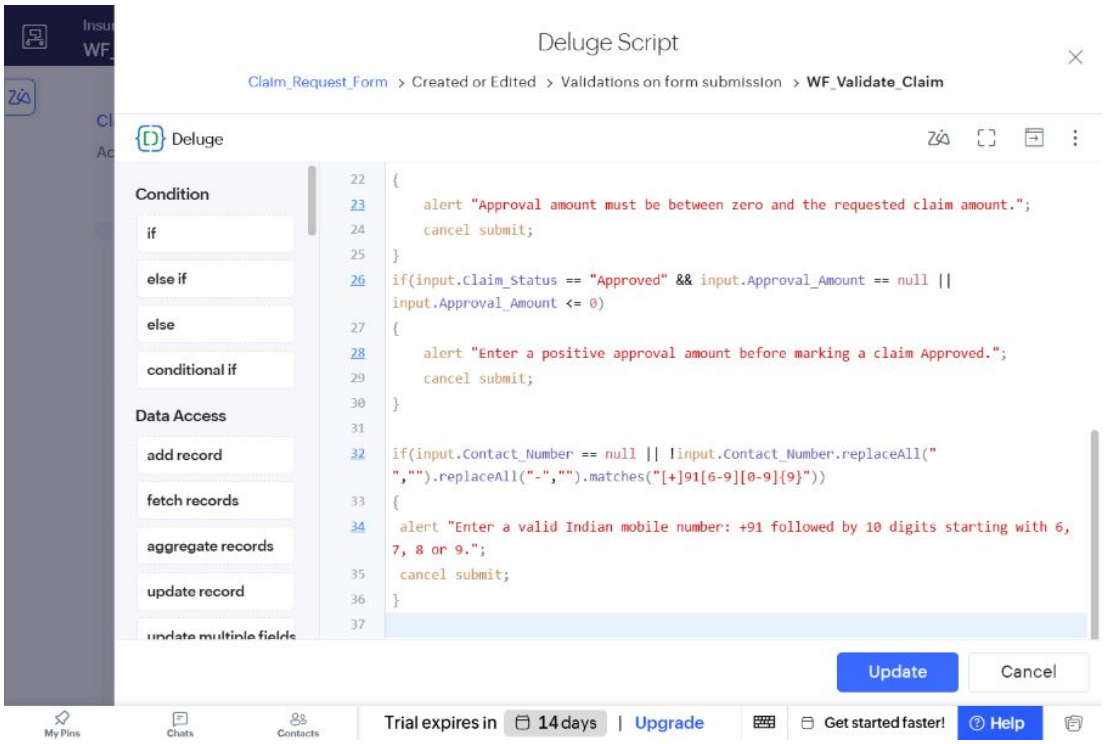


Figure 3. Validation workflow in the Deluge editor; full saved script is printed in Section 5.

10. Implementation Evidence - Notification Code

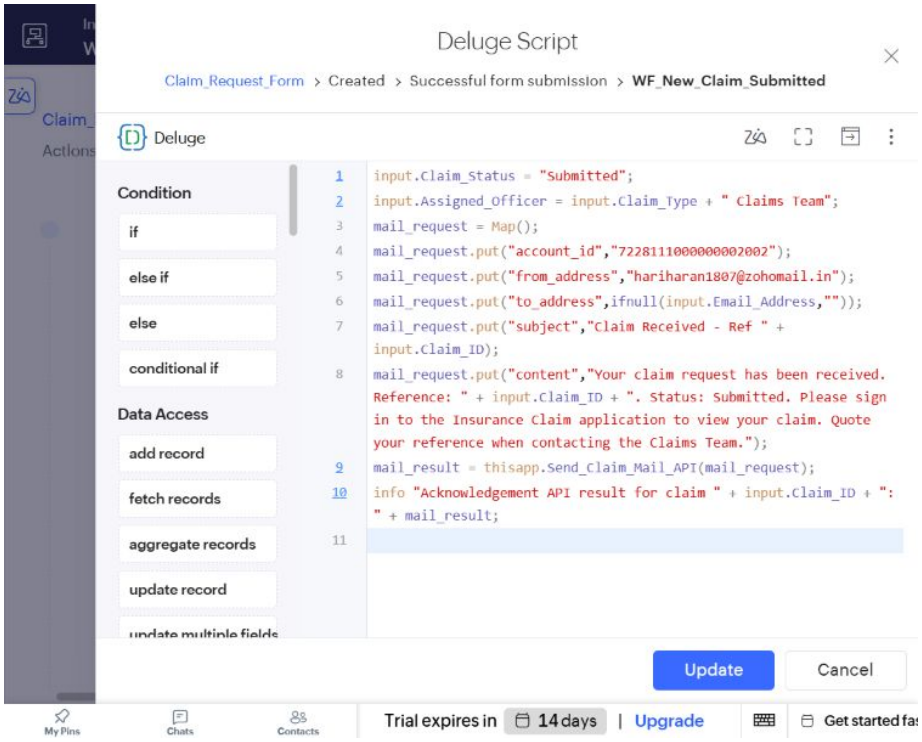


Figure 4. Submission workflow calling the shared mail API function.

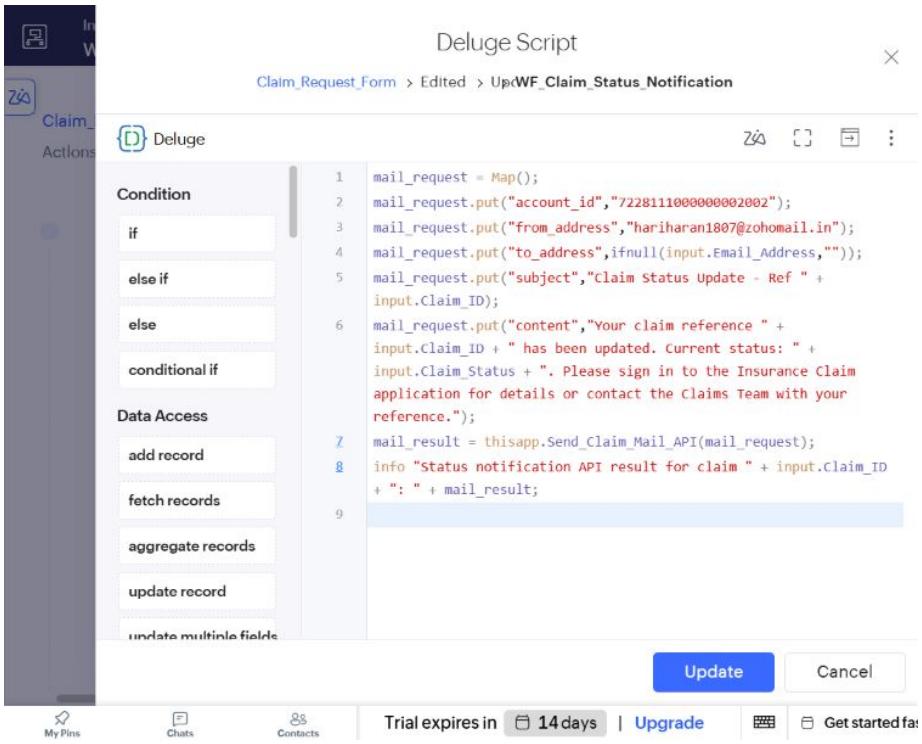


Figure 5. Status-change notification workflow in the Deluge editor.

10. Implementation Evidence - Rejection Tests

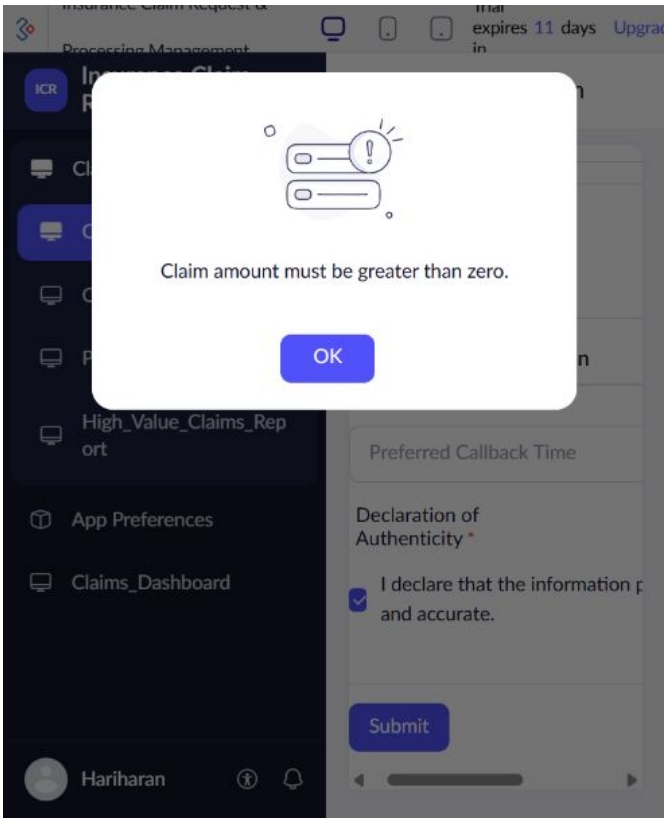


Figure 6. Zero amount rejected.

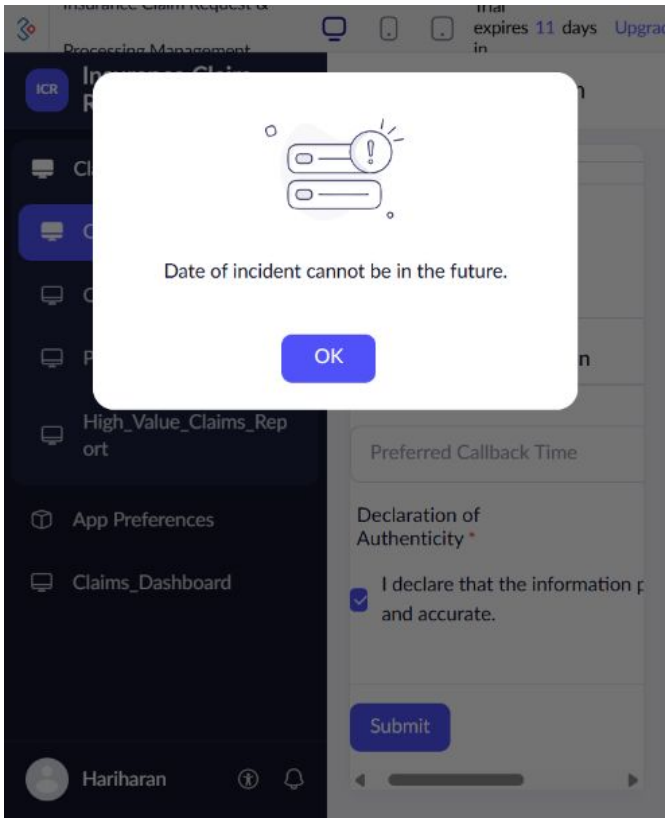


Figure 7. Future incident date rejected.

10. Implementation Evidence - Approval Tests

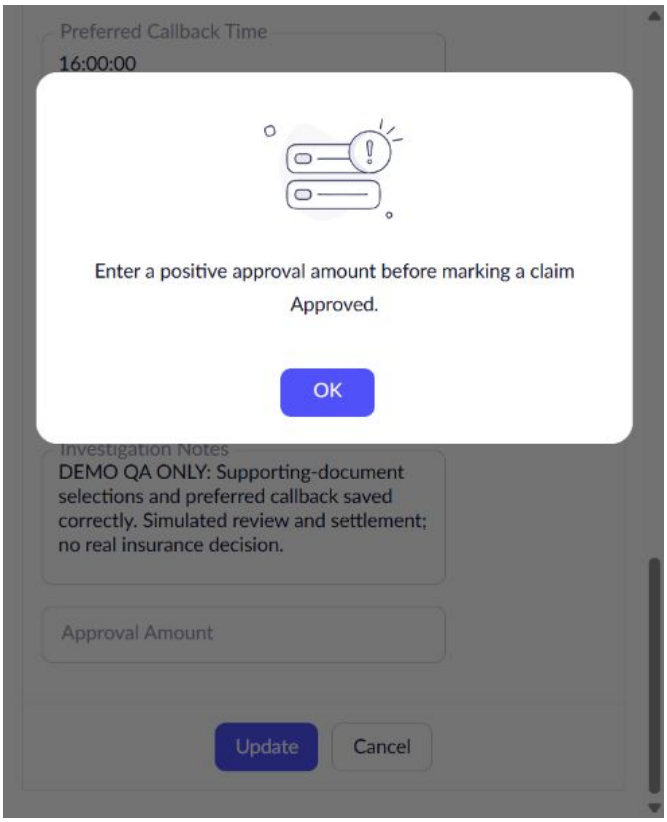


Figure 8. Approval without a positive amount rejected.

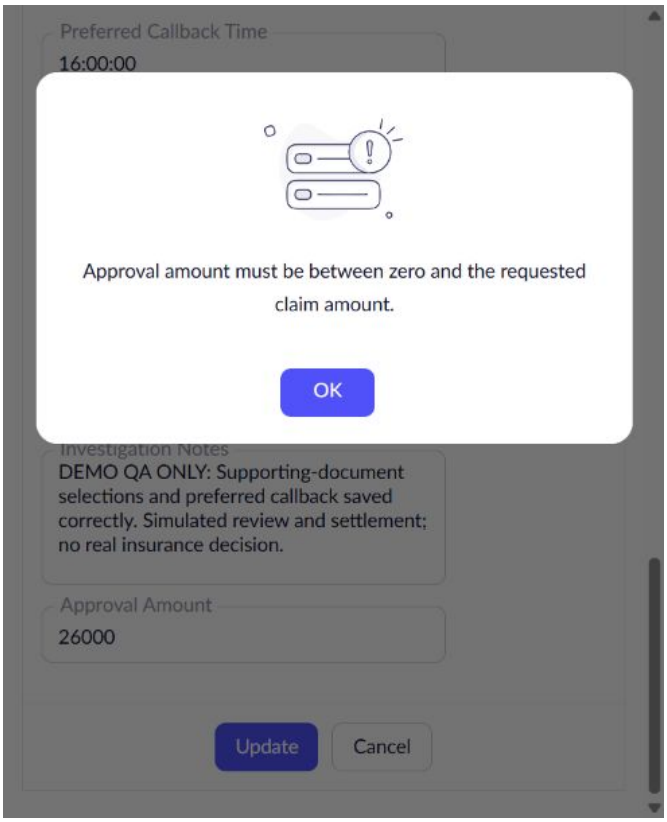


Figure 9. Approval exceeding the requested amount rejected.

10. Implementation Evidence - Execution Logs

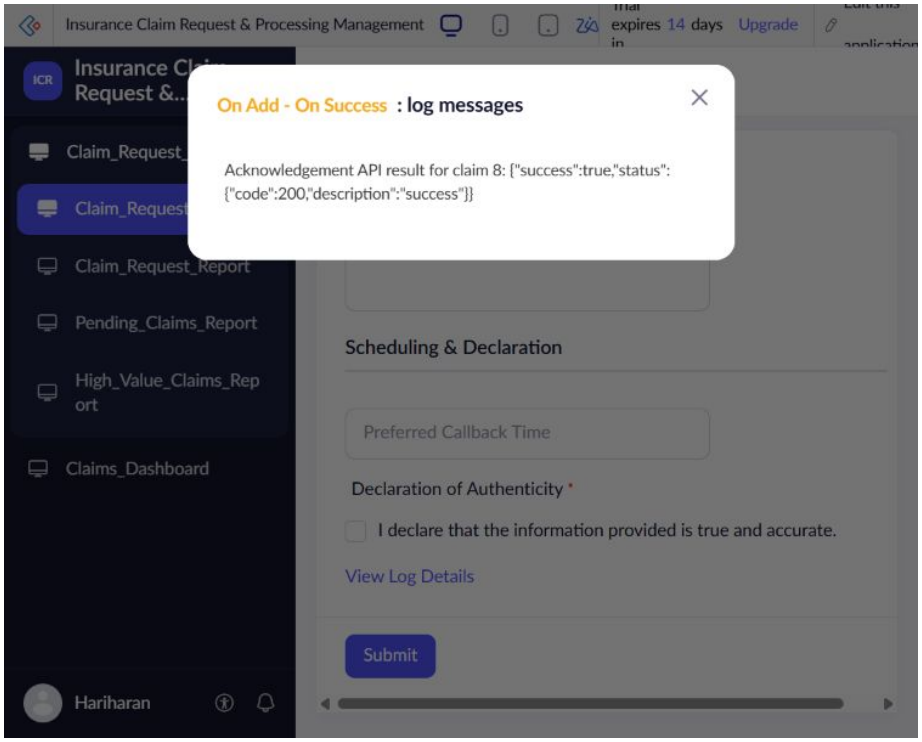


Figure 10. Acknowledgement workflow log shows success and API code 200.

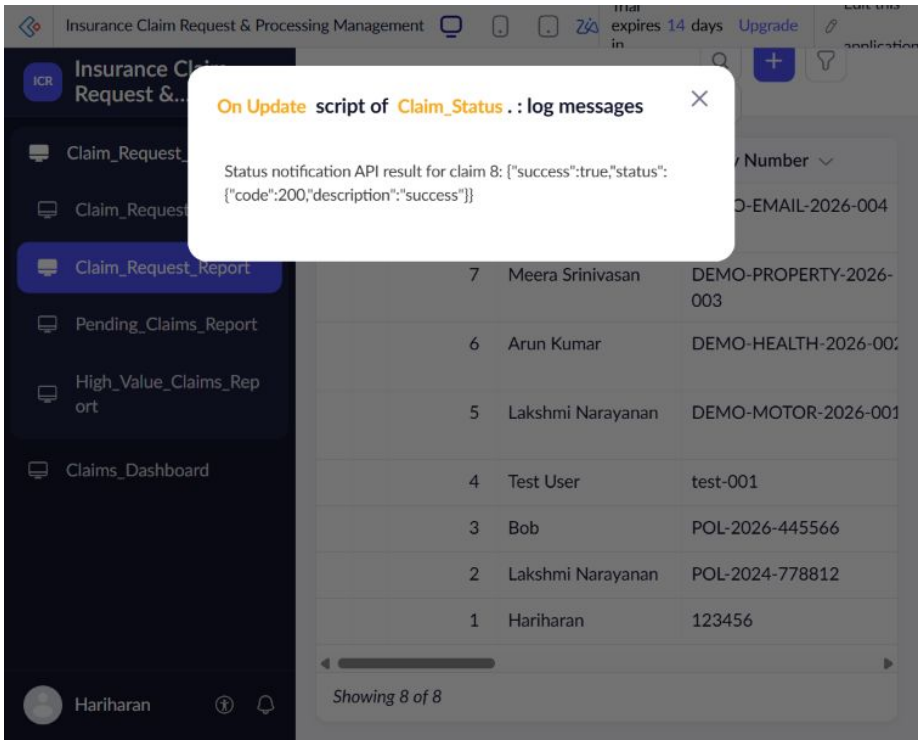


Figure 11. Status workflow log shows success and API code 200.

10. Implementation Evidence - Delivered Output

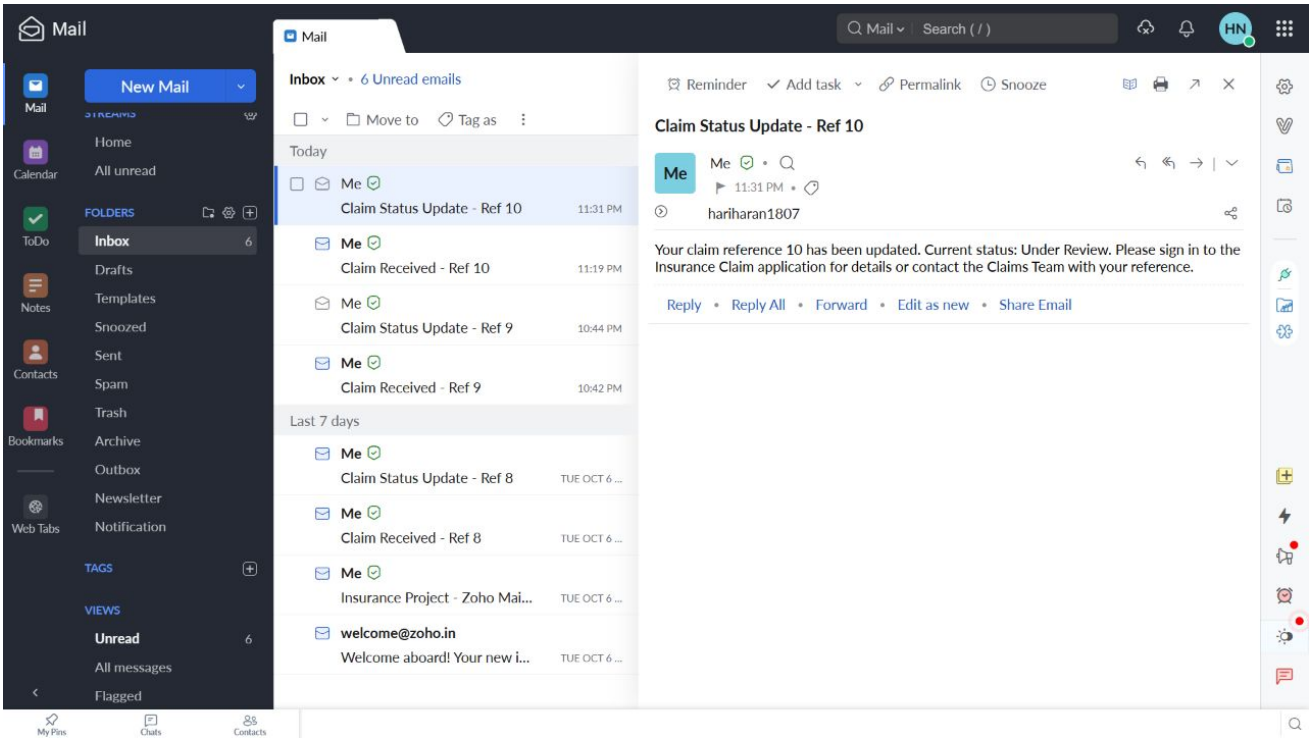


Figure 12. Claim #10 status email received with current status Under Review.

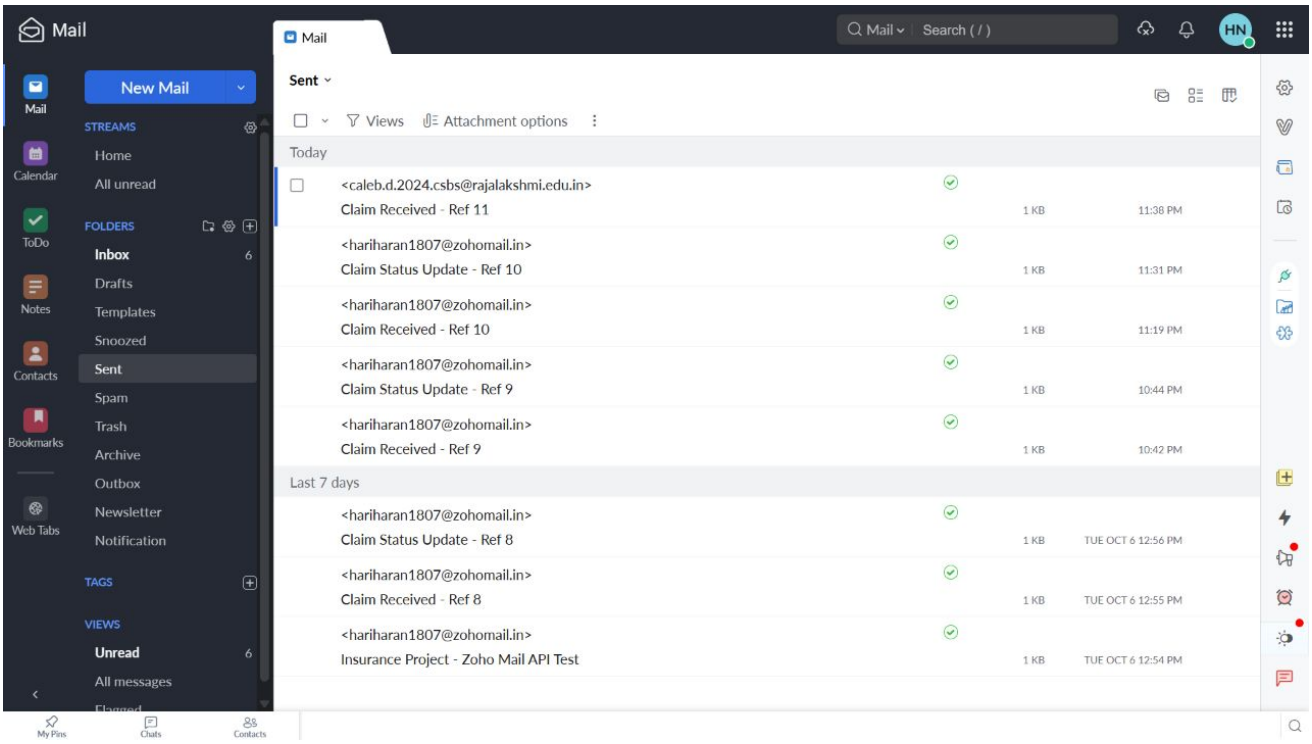


Figure 13. Zoho Mail marks claim #11 acknowledgement and earlier status emails Delivered.

11. Expected Learning Outcomes

1. Use Created, Validation, Success and field-update workflow events.

2. Write Deluge conditions that reject invalid submissions.

3. Combine native references, routing and API notifications.

4. Debug with info output and verify business effects in reports and mail.

12. Result

Four workflows, automatic references, routing, processing validations and notification calls were implemented and tested. Positive and negative tests behaved as documented, including delivery of the latest officer and policyholder notifications.

Conclusion

Automation improves consistency and reduces manual work while retaining a clear staff processing path and traceable claim reference.

Observed scope: The phone rule is scoped to Indian mobile numbers. Mail delivery is verified through Zoho Mail status and recorded inbox evidence; email is optional on the claim form.

Department of Computer Science and Business Systems

Rajalakshmi Engineering College